Python チートシート — 検算・デバッグ用

本番は Java。Python は REPL で電卓・検算・ランダムテスト生成に使う。
計算量は気にしない（O(n)挿入でも、書くのが速ければ勝ち）。

REPL のお作法

- 最終行の式は**自動表示**される → `print()` はほぼ不要
- `_` に**直前の結果**が入る
- `permutations(...)` などイテレータは `list()` で実体化しないと中身が見えない。**一度なめると空**

---

0. 起動テンプレ

```
import sys, math, random, itertools as it
from itertools import permutations, combinations, combinations_with_replacement
from itertools import product, accumulate, groupby, chain, pairwise
from collections import deque, defaultdict, Counter
from functools import lru_cache, reduce, cmp_to_key
from fractions import Fraction
from decimal import Decimal, getcontext
import heapq, bisect

sys.setrecursionlimit(10**7)
getcontext().prec = 50

def show(x): print(list(x))   # イテレータの中身を一発表示
INF = float('inf')
```

---

1. 演算

オーバーフロー検出（Pythonは絶対に溢れないので、Java側で溢れたかの判定に使える）：

```
2**31 - 1        # 2147483647            Java int の上限
2**63 - 1        # 9223372036854775807   Java long の上限
x > 2**63 - 1    # True なら long でも溢れている
```

負数の除算・剰余（Javaと符号が違う）：

```
-7 // 2, -7 % 2   # (-4, 1)  Pythonはfloor方向・%は除数の符号に揃う
                  # Javaは (-3, -1)（0方向切り捨て）
7 / 2             # 3.5   `/` は常にfloat。整数商は `//`
divmod(17, 5)     # (3, 2)  商と余りを同時に
```

```
pow(a, b, m)      # a^b mod m
pow(a, -1, m)     # mod逆元
```

math：

```
math.isqrt(n)                    # 整数平方根（誤差ゼロ）
math.gcd(a, b, c); math.lcm(a, b, c)
math.comb(n, r); math.perm(n, r); math.factorial(n)
math.hypot(dx, dy); math.dist(p, q); math.atan2(y, x); math.pi
math.floor(x); math.ceil(x); round(x, 3)
```

ビット・進数：

```
bin(13), hex(13)         # '0b1101', '0xd'
f'{13:08b}'               # '00001101'（ゼロ埋め2進）
int('1101', 2)            # 13（基数指定で読む）
n.bit_count()              # 立っているビット数（3.10+）
n.bit_length()             # ビット長
x & -x                     # 最下位の1ビット
```

厳密計算（幾何・確率の検算）：

```
Fraction(1, 3) + Fraction(1, 6)   # Fraction(1, 2)  誤差ゼロの分数
Decimal(1) / Decimal(7)           # 任意精度小数（prec桁）
```

---

2. データ構造とスライス

リスト

```
a = [0] * n                          # 初期化
g = [[0]*w for _ in range(h)]        # 2次元。[[0]*w]*h は参照共有で禁止
a.sort(); a.sort(key=lambda t: (t[0], -t[1]))   # 第1昇順・第2降順
sorted(a)                            # 非破壊版
a.index(x); a.count(x); x in a
sum(a), max(a), min(a), len(a)
```

スライス `a[start:stop:step]`

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

a[2:5]       # [2, 3, 4]     stopは含まない。a[i:i+k]で「iからk個」
```

```
a[:3]        # [0, 1, 2]
a[-3:]       # [7, 8, 9]     末尾3つ
a[:-3]       # 末尾3つを除く
a[:]         # 浅いコピー（b = a は同一参照なので注意）

a[::2]       # [0,2,4,6,8]   1つおき
a[::-1]      # 反転
a[5:1:-1]    # [5, 4, 3, 2]  逆向き範囲

a[5:100]     # [5,...,9]     範囲外はエラーにならずクランプ
a[2:5] = [20, 30]   # 置換（長さが違ってもOK）
a[1:1] = [97]       # 挿入
del a[2:5]           # 範囲削除

"abcdef"[::-1]       # 'fedcba'  文字列も同じ（代入は不可）
```

set / dict

```
s, t = set(a), set(b)
s & t, s | t, s - t, s ^ t     # 積/和/差/対称差 – 答えの突き合わせに便利
x in s                          # O(1)

d = {}
d[k] = d.get(k, 0) + 1          # キー無しでも安全に集計
sorted(d.items(), key=lambda kv: kv[1])   # 値でソート
seen = set(); seen.add((x, y))  # タプルはキーにできる（座標管理）
```

---

3. itertools

返り値はイテレータ。**中身を見るなら `list(...)` か `show(...)`**。

```
list(permutations([1,2,3]))       # 全順列
list(combinations(a, 2))          # 全ペア（nC2）
list(combinations_with_replacement(a, 2))   # 重複組合せ
list(product([0,1], repeat=n))    # 2^n の全ビット列挙
list(product(range(3), repeat=4)) # 各桁0-2の全パターン

list(accumulate(a))               # 累積和
list(accumulate(a, initial=0))    # 先頭0付き
list(accumulate(a, max))          # 累積max

list(chain.from_iterable(g))      # 2次元を平坦化
[(k, len(list(v))) for k, v in groupby('aaabbc')]   # ランレングス
list(pairwise(a))                 # 隣接ペア（3.10+）
```

全探索での検算の定番：

```
best = min(cost(p) for p in permutations(a))
```

---

4. collections

```
q = deque([1, 2, 3])              # 両端O(1)。BFS・スライド窓
q.append(4); q.appendleft(0); q.pop(); q.popleft()
q.rotate(1)                       # 右回転

g = defaultdict(list); g[u].append(v)    # 隣接リスト（キー初期化不要）
cnt = defaultdict(int); cnt[x] += 1

c = Counter('abracadabra')        # 頻度集計の決定版
c.most_common(2)                  # 上位2件
c['z']                            # 無いキーは0（例外にならない）
c1 + c2, c1 - c2, c1 & c2         # 多重集合の加減・min
Counter(a) == Counter(b)          # アナグラム／多重集合一致の判定
```

---

5. heapq / bisect

```
heapq.heapify(h)                  # その場でヒープ化
heapq.heappush(h, x); heapq.heappop(h)
h[0]                              # peek（min-heap）
heapq.heappush(h, -x)             # max-heap は符号反転で
heapq.heappush(h, (dist, node))   # タプルで優先度（Dijkstra）
heapq.nlargest(3, a); heapq.nsmallest(3, a)   # ソート不要で上位k
```

```
a = [1, 3, 3, 5, 7]               # ソート済みが前提
bisect.bisect_left(a, 3)          # 1
bisect.bisect_right(a, 3)         # 3 → 「3以下の個数」
a[bisect.bisect_left(a, x)]       # x以上の最小
a[bisect.bisect_left(a, x) - 1]   # x未満の最大
bisect.insort(a, 4)               # 順序を保って挿入
```

---

6. 便利構文

```
a, b = b, a                       # スワップ
first, *rest = [1, 2, 3, 4]       # 1, [2,3,4]
if 0 <= x < H and 0 <= y < W: ... # 連鎖比較（範囲チェック）
v = a if cond else b              # 三項演算子
```

```
for i, x in enumerate(a, start=1): ...   # 番号付き（1始まり）
for x, y in zip(xs, ys): ...
list(zip(*mat))                  # 行列の転置

[x*x for x in a if x > 0]        # 内包表記
sum(1 for x in a if x > 0)       # 条件カウント
any(x < 0 for x in a); all(x > 0 for x in a)

a.sort(key=cmp_to_key(lambda x, y: x - y))   # Comparator感覚で書ける

ord('a'), chr(97)                # 97, 'a'
f'{x:.10f}'  f'{n:05d}'  f'{n:,}' # 小数10桁 / ゼロ埋め／桁区切り
print(f'{n=}, {arr=}')           # `x=値` の形で出るdebug用（3.8+）

@lru_cache(maxsize=None)         # メモ化＝愚直DPを1行で
def f(i, rest): ...
f.cache_clear()                  # ケースごとにリセット

for x in a:
    if bad(x): break
else:
    print('breakせず完走')        # for-else
```

---

7. ランダムテスト生成

```
random.seed(42)                  # 再現用
random.randint(1, 10)            # 1〜10 両端を含む（Javaの乱数と感覚が違う）
random.randrange(10)             # 0〜9
random.choice('RPS'); random.shuffle(a)
random.sample(range(100), 5)     # 重複なしで5個
''.join(random.choices('ab', k=8)) # ランダム文字列（重複あり）

# テストケースを1つ作って標準入力形式の文字列にする例
n = random.randint(1, 8)
a = [random.randint(1, 20) for _ in range(n)]
case = f"{n}\n{' '.join(map(str, a))}\n"
print(case)

# 複数ケースをまとめて生成
cases = []
for _ in range(20):
    n = random.randint(1, 8)
    a = [random.randint(1, 20) for _ in range(n)]
    cases.append(f"{n}\n{' '.join(map(str, a))}\n")
```

よくあるハマり

- `randint(a, b)` は**上端を含む**

- 負の `//` `%` はJavaと符号が違う（Pythonはfloor）
- `[[0]*w]*h` は全行が同一参照 → `[[0]*w for _ in range(h)]`
- `b = a` は参照。コピーは `a[:]` / `a.copy()`、2次元は `copy.deepcopy`
- イテレータは一度きり。見るなら `list()` で固める
- `int(n**0.5)` は誤差で1ずれうる → `math.isqrt(n)`
- `/` は必ずfloat（`10/2` は `5.0`）。整数が欲しければ `//`